24. LinkedList in Java — Quick Notes

Definition

- Implements `List` interface
- Implements `Deque` interface
- Internally uses: Doubly Linked List

LinkedList Hierarchy

Collection
↓
List Queue
↘ ↓
Deque
↓
LinkedList

Important

LinkedList can be used as: * List, * Queue, * Deque

Why LinkedList Supports Deque?

Because doubly linked list maintains:
Head ⟷ Node ⟷ Node ⟷ Tail

Hence:

- Insert at front → O(1)
- Insert at end → O(1)
- Delete from front → O(1)
- Delete from end → O(1)

LinkedList vs ArrayList

Feature	LinkedList	ArrayList
Internal DS	Doubly Linked List	Dynamic Array
Random Access	O(n)	O(1)
Insert at End	O(1) Worst	O(1) Amortized
Insert at Beginning	O(1)	O(n)
Remove First	O(1)	O(n)
Memory	No unused capacity	May have extra capacity
Cache Friendly	❌	✅

Advantages of LinkedList

1. No Wasted Capacity

If there are: n elements
Then memory is used only for: n nodes
Unlike ArrayList: No unused internal array space.

---

2. No Need for Contiguous Memory

Nodes can be stored: Different Memory Locations
Unlike ArrayList: No large contiguous memory block needed.

---

3. Worst Case O(1) End Insertions

list.add(x);
LinkedList: O(1) Worst Case
ArrayList: O(1) Amortized
because resizing may occur.

Creating LinkedList

```
LinkedList<Integer> list = new LinkedList<>();
```

Import:

```
import java.util.LinkedList;
```

Basic Example

```
LinkedList<Integer> list = new LinkedList<>();

list.add(10);
list.add(20);
list.add(30);

System.out.println(list);
```

Output: [10, 20, 30]

Common List Methods

Method	Purpose
add(x)	Add at end
add(i,x)	Insert at index
contains(x)	Check existence
remove(i)	Remove at index
get(i)	Get element

set(i,x)	Replace element
indexOf(x)	First occurrence
lastIndexOf(x)	Last occurrence
isEmpty()	Check empty

Queue Operations

Functionality	Throwing Method	Non-Throwing Method	Failure Behavior	Purpose
Add Element	`add(x)`	`offer(x)`	`add()` throws exception, `offer()` returns `false`	Insert element at rear/end
Remove Front Element	`remove()`	`poll()`	`remove()` throws exception, `poll()` returns `null`	Remove element from front
Access Front Element	`element()`	`peek()`	`element()` throws exception, `peek()` returns `null`	Retrieve front element without removal

---

Deque Methods

Throwing Method	Non-Throwing Counterpart	Failure Behavior	Purpose
`addFirst(x)`	`offerFirst(x)`	Returns `false`	Add element at beginning
`addLast(x)`	`offerLast(x)`	Returns `false`	Add element at end
`removeFirst()`	`pollFirst()`	Returns `null`	Delete first element
`removeLast()`	`pollLast()`	Returns `null`	Delete last element
`getFirst()`	`peekFirst()`	Returns `null`	View first element
`getLast()`	`peekLast()`	Returns `null`	View last element

LinkedList: Equivalent Queue and Deque Methods

Queue Method	Equivalent Deque Method	Meaning
`add(x)`	`addLast(x)`	Insert at rear
`offer(x)`	`offerLast(x)`	Insert at rear
`remove()`	`removeFirst()`	Remove front element
`poll()`	`pollFirst()`	Remove front element
`element()`	`getFirst()`	Access front element
`peek()`	`peekFirst()`	Access front element

Time Complexity:

Interface	Operations	Time Complexity
List	**`add(x)`, `isEmpty()`**	**O(1)**
List	**`add(index,x)`, `get(index)`, `set(index,x)`, `remove(index)`**	**O(index)**
List	**`contains(x)`, `indexOf(x)`, `lastIndexOf(x)`**	**O(n)**
Queue	**`add()`, `offer()`, `remove()`, `poll()`, `element()`, `peek()`**	**O(1)**
Deque	**`addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `peekFirst()`, `peekLast()`, `offerFirst()`, `offerLast()`, `pollFirst()`, `pollLast()`**	**O(1)**

Important

- `get()` and `set()` are slower than ArrayList because LinkedList requires traversal from head or tail.
- LinkedList is best used as Queue/Deque.
- For frequent random access, prefer ArrayList.

When to Use LinkedList?

Use LinkedList When:

- Frequent insertions/deletions at ends
- Need Queue implementation
- Need Deque implementation
- Need worst-case O(1) end operations

Prefer ArrayList When:

- Frequent random access
- More reads than writes
- Better cache performance needed

Important Quick Points

Topic	Point
Internal DS	Doubly Linked List
Implements	List, Queue, Deque
Random Access	O(n)
End Insert/Delete	O(1) Worst Case
Cache Friendly	No
add()	Same as addLast()
poll()	Same as pollFirst()
peek()	Same as peekFirst()
Best Use	Queue/Deque operations

LinkedList Insertion

Difficulty: **Basic**Accuracy: **77.23%**Submissions: **7K+**Points: **1**

Given an integer array **arr[]**. The task is to insert elements of given array to an LinkedList and return it.

Example 1:

Input: arr[] = [1, 2, 3, 4]
Output: 1 2 3 4
Explanation: Simply insert into LL and return it.
Input: arr[] = [3, 2, 1]
Output: 3 2 1
Explanation: Simply insert into LL and return it.

Constraints:

1 <= arr.length <= 1000

1 <= arr[i] <= 1000

```
// User function Template for Java
class Solution {
    public static LinkedList<Integer> insertion(int arr[]) {
        // Your code here
        LinkedList<Integer> list = new LinkedList<>();
        for(int i=0;i<arr.length;i++){
            list.add(arr[i]);
        }
        return list;
    }
}
```

LinkedList Traversal

Difficulty: **Basic**Accuracy: **69.32%**Submissions: **8K+**Points: **1**

Given an LinkedList **list** that contains integer elements. The task is to iterate through the given list and print its elements.

Examples:

Input: list = [1, 2, 3, 4]
Output: 1 2 3 4
Explanation: Simply iterate through the ll and print the elements.
Input: list = [3, 2, 1]
Output: 3 2 1
Explanation: Simply iterate through the ll and print the elements.
Constraints:

1 <= list.size() <= 1000

1 <= list[i] <= 1000

```
/*
class Node {
    int data;
    Node next;

    Node(int val) {
        data = val;
        next = null;
    }
}
*/

class Solution {
    void printList(Node head) {
        // code here
        Node curr = head;
        while(curr!=null){
            System.out.print(curr.data + " ");
            curr = curr.next;
        }
    }
}
```